

Encryption Isn't Enough: Supervised Activity Inference on Drone Telemetry

Abdulahim Ünver and Kalle Mossberg

Abstract—This study investigates eavesdropping side channel attacks on drones using the MAVLink telemetry protocol, by applying traffic analysis of the payload size and timing intervals of encrypted communication. By training a supervised model on data acquired by simulating UAV missions on the PX4 framework, focusing on size and timing related features, different models capable of detecting events of interest has been produced. It was found that the strongest model achieved an overall accuracy of 72% in classification, where some events such as arming and take-off had perfect accuracy. Size related features had the biggest importance in classifying events, which implies that the vulnerability of MAVLink derives from the presence of distinct payload size. Our work shows that telemetry protocols with distinct traffic behaviour are vulnerable even if the payload data is encrypted.

Sammanfattning—Denna rapport undersöker avlyssningsattacker mot drönare som använder telemetriprotokollet MAVLink, detta genom trafikanalys av payloadstorlek och tidsintervall av meddelanden i krypterad telemetridata. Genom att träna en supervised learning-modell på data som samlats in genom simulering av UAV-uppdrag på PX4-ramverket, detta med fokus på storlek- och tidsintervallkaraktäristik, har flera modeller kapabla av att detektera specifik drönaraktivitet av intresse tränats. Detta har resulterat i att en modell med träffsäkerhet på 72% i klassifiering av händelser skapats, där vissa händelser som armering och start klassificeras helt korrekt. Storleksrelaterad karaktäristik har visat sig vara av störst vikt vid klassifiering, vilket antyder att sårbarheten hos MAVLink härstammar från dess distinkta meddelandestorlek. Vårt arbete visar att telemetriprotokoll med fördefinierad struktur är sårbara även med krypterad meddelandetrafik.

Index Terms—(MAVLink, AV, Autonomous drone, Traffic analysis, eavesdropping, AI)

Supervisors: Axel Andersson

TRITA number: TRITA-EECS-EX-2026:145

I. INTRODUCTION

The adoption and usage of drones is rapidly increasing, both in warfare and for civilian applications. An interesting branch of drone application is the autonomous vehicle (AV); a vehicle that can act without the direct control of a human. These vehicles can be applied in the sea, ground or air domain, with the common link that they rely solely on sensory data and software to accomplish a goal or mission. In the context of this thesis the focus will be on aerial autonomous drones. These systems usually are composed of a drone, pilot software and a ground control station (GCS). In this study, this is composed of a simulated quadcopter running PX4-autopilot software [1], connected to QGroundStation [2] and communicating over the MAVLink protocol [3]. Since the pilot system primarily

relies on sensory data rather than user control input, most of the communication between the pilot software and GCS is generally aimed at informing the user of the status and location of the drone. This communication channel can be encrypted, but if there exists variations in in communication intensity and size depending on the drones current activity, an adversary could gain valuable information without breaking the encryption. For example, an adversary may want to find points of low attack hardness [4].

The focus of this Bachelors thesis is therefore to:

- Investigate if valuable information can be gained through traffic analysis of encrypted MAVLink communication.
- Analyze which properties of MAVLink communication is usable to develop an eavesdropping attack.
- Develop an eavesdropping attack on an aerial drone running an encrypted lightweight communication protocol.

II. BACKGROUND

A. MAVLink

The Micro Aerial Vehicle Link (MAVLink) protocol is a lightweight serialization protocol designed for communication between ground stations and unmanned vehicles. This means that MAVLink serializes messages and commands in a specific predefined binary format, making it lightweight by its inherent minimal overhead [5]. MAVLink exists in version 1 & 2, where version 2 is the current recommended version and what will be referred to as MAVLink hereafter.

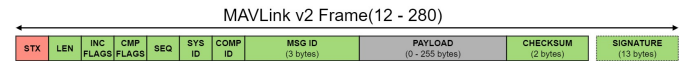


Fig. 1. Structure of a MAVLink message [6]

A MAVLink message structure (Fig. 1) is based on a header and a payload, where the header carries information about the message and the payload the data transported by the message. The header is appended to the payload, and is needed to correctly de-serialize the data. The message frame is built on 10 distinct fields, with an optional 11th signature field. The first field is STX, which refers to the symbol that represents the start of the MAVLink frame. In the case of MAVLink 2.0, it is 0xFD. The second byte, or second field, is LEN which represents the message length in bytes. Third and fourth field are Incompatibility flags and Compatibility flags, which indicate if there are any features in the message that need to be accounted for when parsing the message. Examples of such features could be that the message is signed or that

the message carries a priority rating. The fifth through eighth field is the sequence number (SEQ), the System ID (SYSID) and the Component ID (COMPID). The SEQ increments for every message, and makes message losses identifiable. SYSID identifies the message sender. The next fields are the message ID (MSGID) and Payload, which identifies the message type and message. The last field is the checksum, a hashed value based on the bytes of the message that is used to verify that the message has not been corrupted in any way during transmission. A signature field can also be appended, consisting of a link ID, a timestamp and an encoded signature key to verify the source of the message. An important detail to remember for this thesis is that a MAVLink message is fixed in size for every field except Payload.

MAVLink messages are categorized into two classes: state messages and command messages. State messages originate from the unmanned vehicle and contain information about the state of the system such as position or velocity, and command messages originate from the GCS and contain a command or a mission plan. The number of different MAVLink messages is too numerous to be contained within the scope of this thesis, with the standard library covering more than 200 different messages, but a few examples of the more prevalent or important will given when applicable. Since MAVLink currently does not have a native end-to-end message encryption protocol, these messages are not encrypted [7].

B. Related Work

This thesis is primarily based on the security concerns raised by Vekler et al. [7] regarding the MAVLink protocol. The authors of this study focus on covert channels that leak sensitive data through the MAVLink header while avoiding detection by statistical analysis. This is done by compromising the drone firmware to establish a malicious covert channel for data leakage, while keeping the message entropy unchanged to avoid warden detection. This thesis applies their line of reasoning regarding network warden detection avoidance to perform an eavesdropping attack.

Utilizing machine learning, or deep learning [8] to perform traffic analysis on encrypted traffic data is already well explored. However, our work demonstrates that MAVLink leaks enough information for classification of UAV behavior to be possible. Our work also explores which specific vulnerabilities of MAVLink pose a critical role in leaking information regarding drone activity.

C. Classification with Machine Learning

Machine learning and deep learning models [9] can be used to classify, for example, events or images by learning the structure and relationships between the data and the different classes. ML models are developed by using training data sets to develop an understanding of different patterns. Supervised Learning for classification involves training a model that observes data through different metrics or properties called features, and then providing labels which is an answer sheet, for the model. Unsupervised learning models are not provided any labels for the dataset, and classify the data into clusters

from the patterns it observes. Since both Supervised and Unsupervised Learning require the development of features, understanding of the classification is problematic, as some features may have a stronger importance or correlation to classify the data. Permutation feature importance is computed by taking a trained model and a validation set, then for each feature, randomly permuting its values across samples to break the association with the target while keeping all other features fixed, and measuring the increase in loss. Cross-entropy loss is a common loss metric that measures how far the model's predicted class probabilities are from the true labels. It penalizes the model strongly when it assigns low probability to the correct class, so minimizing cross-entropy encourages more confident and correct predictions.

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log(\hat{p}_{i,k}), \quad (1)$$

where N is the number of samples, $y_{i,k}$ is 1 if sample i belongs to class k and 0 otherwise, and $\hat{p}_{i,k}$ is the predicted probability for class k .

Classification can yield wrong answers, such as false negatives (FN), or false positives (FP), but also true positives and negatives (TP and TN). One way to evaluate the general accuracy of a model is through an F1 score.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4)$$

$$\text{F1-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5)$$

A loss function is used during training to measure how wrong the model's predictions are compared to the true labels. Let $\mathbf{x}_i$ be the feature vector for sample i , y_i its true class label, and $\hat{\mathbf{p}}_i = f_{\theta}(\mathbf{x}_i)$ the probability distribution predicted by the model. Here, θ represents the model parameters.

For multi-class classification, a common loss function is cross-entropy loss:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \mathbf{1}(y_i = k) \log(\hat{p}_{i,k}), \quad (6)$$

where N is the number of training samples, K is the number of classes, $\hat{p}_{i,k}$ is the predicted probability that sample i belongs to class k , and $\mathbf{1}(y_i = k)$ is 1 if the true label is class k and 0 otherwise. A lower loss means that the model assigns a higher probability to the correct class.

If the accuracy while training the data yields high differences in testing the model with new data, it could imply that the model is overfit. This means that the model is fitting too much into its training set. This can stem from that noise in the data is interpreted as patterns or that a model is overly complex with too many parameters. This can be mitigated by using more training data, or by addressing the underlying issues previously mentioned.

III. PROBLEM FORMULATION

To accomplish the goals set in the introduction, a defined problem scenario must be set. In the case of this thesis, it will be that of an adversary trying to deploy an eavesdropping attack on an aerial autonomous drone using encrypted telemetry communication. The goal of this adversary is to gain actionable intelligence on the drones current activity. Therefore, the objective of this thesis becomes to research the feasibility of this scenario. This can be broken down into three parts:

- Data needs to be collected to represent what an adversary would have access to.
- A decision on what drone activity (hereafter referred to as "events") would be of interest to an adversary must be made.
- Research if an adversary could gain this information through the data available by training a supervised model must be conducted.

To accomplish these objectives, certain problems must be solved. These are as listed below:

1) *Simulation*: To be able to investigate how MAVLink works in a practical setting, a simulated environment for drone operations must be set up. This simulation needs to be realistic enough to represent a real-world scenario.

2) *Data Processing*: To conduct traffic analysis of the MAVLink communication, this data must be saved and processed from its raw binary form into a readable and usable message form. For the data to be usable, it needs to be timestamped in such a way that it can be related to the drones current event when it was sent.

3) *Labeling*: The saved and processed messages need to be sorted and labeled in relation to the drones current event when sent, in order for the data to be usable for traffic analysis or a learning model.

4) *Traffic analysis*: To effectively train a supervised model on the labeled data, an analysis of which features are of importance must be done.

5) *Model training*: Based on traffic analysis, a model needs to be trained to verify if the problem scenario is feasible.

A. Adversary Model and Constraints

We assume that the adversary can passively capture the message packets that are wirelessly transmitted when a drone is communicating with the ground control station. The adversary has acquired a drone similar to the one being observed, with which he has flown and observed a number of missions. By eavesdropping on these missions he can gather information on the payload data size, either directly or through it being directly correlated to the packet size, and the timestamps of each packet. The adversary can then label each packet to an event in order to train a supervised model. Afterwards, the adversary can apply this model on a target drone by eavesdropping. Doing this, he can utilize the model to infer what a drone with the same protocol is doing without observing the drone itself. We also assume the adversary does not know the full semantics of MAVLink, therefore making the model applicable to similar communication protocols. This is accounted for

as our methodology does not use any specific knowledge of MAVLink, such as what specific payload size certain messages have.

The data collected are purely simulated and not acquired from real-world testing. Therefore, there is an inherent risk of unknown and unaccounted for variables that could affect the result of a real-world application. Another limitation could be in the assumptions made on the encryption of the MAVLink messages, since the models only take payload length and timestamps into account. We assume that the encryption used does not interfere with the inherent statistical features of the message stream. Additionally this thesis only focuses on single drone analytics. This as swarm drone activity would require the adversary to be able to differentiate the data stream into individual streams based on the targeted recipients, which falls outside of the scope.



Fig. 2. Screenshot from Gazebo

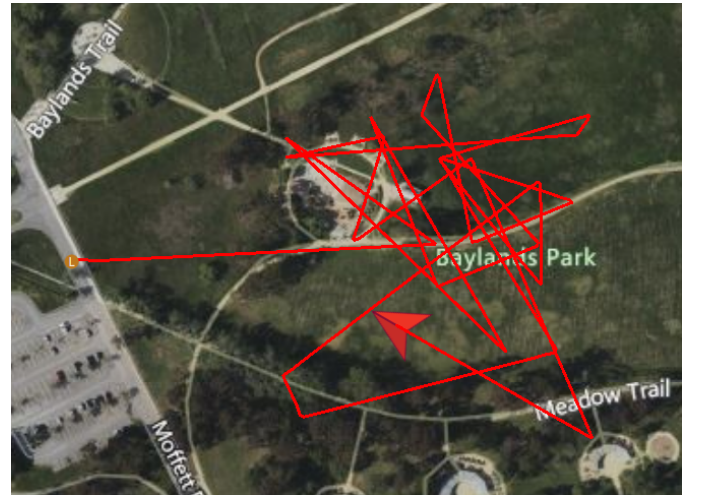


Fig. 3. Screenshot from QGroundControl

IV. METHODOLOGY

A. Simulation Environment

To create a simulated environment PX4 Autopilot [1], Gazebo [10], QGroundControl [2] (QGC) and MAVSDK [11] is used. PX4 is an open-source flight control stack for

AV, and supports multiple types of vehicles and integration with other software over the MAVLink protocol. PX4, as an autopilot, is what actually controls the drone and translate the MAVLink missions into servo control input. Gazebo is a physics simulation environment for robotics development (Fig. 2). Together with QGroundStation (Fig. 3), a ground control station, this constitutes the simulation environment. A ground control station is a graphical interface for interaction with autopilot systems, and can be used for mission planning, real-time flight visualization, telemetry monitoring, etc. By using the built in software-in-the-loop function in PX4 it will pilot a drone simulated in Gazebo. To command the drone, QGroundControl is coupled over a UDP port to the PX4-Gazebo simulation. Since the UDP protocol does not include a built-in verification of packet delivery [12], messages can be lost. Therefore MAVLink still has to handle message losses and repeat lost messages, as in a real-world scenario. By logging all communication passed through the UDP port, all data packets that has been sent can be saved in its raw binary format.

B. Events of importance

To be able to collect relevant data using the simulation environment, the decision of what constitutes relevant data must be done. This is based on what events are deemed to be of interest to an adversary. For the purpose of this thesis, the events chosen are:

- Takeoff: Drone takeoff from a stationary ground position
- Landing: Drone landing from a hovering position.
- Return to launch (RTL): Returning from the current position to the recorded original takeoff location.
- Checkpoint reached: A mission-defined part of the flight path has been reached.
- Arming: The transition from an inactive state into an operational state in which the motors are enabled and the vehicle is allowed to fly.
- Executing mission: The drone starts on the flight path determined by its mission.

With an “other” event added to include all events and communication deemed not to be of interest.

C. Data collection

To use this simulated environment, drone missions need to be created and uploaded to the drone. This can be done manually through QGC, but for repeatability and automation purposes, this has been done with the MAVSDK software development kit which makes programming mission possible in either C++ or Python. To collect relevant data, four main strategies has been used to program tailored missions for specific data. To gather data a serpentine search program, a manually written flight mission, a cyclic takeoff and land-mission and a random objective mission has been written. The purpose of the serpentine search is to effectively gather data on the “checkpoint reached” event, by flying in a tight serpentine shape and covering a large amounts of checkpoints in short time. The random objective missions (as seen in Fig. 3) is also

```

9:53:34.060340 IP localhost.14580 > localhost.14540: UDP, length 14
0x0000: 4500 002a 3761 4000 4011 0560 7f00 0001  E...7a0@...
0x0010: 7f00 0001 38f4 38cc 0016 fe29 fd02 0000  ....8.8....
0x0020: f101 01f5 0000 0001 f28e

```

Fig. 4. Example of a captured packet displayed from the PCAP file.

aimed to gather checkpoint data, but with added randomness in the time domain to avoid creating time dependent patterns in the event data. Both automatically construct missions based on an amount of wanted mission points - the serpentine search creates a grid pattern and the random mission creates random mission points within a predetermined distance from an origin point. The takeoff and land-mission is targeted towards collecting data of takeoff and landings, and consists of multiple repeated takeoffs and landings. The manual written mission is a “normal-use case” mission with five hard-coded mission points, aimed to be able to be used as a reference to make sure the model learns to interpret the data and not to predict the automatically constructed missions.

D. Labeling and Parsing

The simulated environment is handled by an overhead program which handles the bootup sequence, running, and closing of the simulation environment. This process is automated to eliminate any user interference in data collection. Furthermore, the overhead program is responsible for timestamping events to make labeling collected data possible. The timestamping works by reacting to certain terminal output, such as “checkpoint reached”, and timestamping it with the current time. The timestamp with an appropriate label is then saved. The label given to the timestamp is determined by the PX4 terminal output, with each event type having its own keyword - for example, the PX4 terminal output “checkpoint reached 11/15” would result in a “Checkpoint reached” label. At the end of a mission run all timestamps are saved to a CSV file to be used by the analyzer program. To create a universal startpoint for the timestamps and datastream a specific MAVLink message is used, `PARAM_REQUEST_READ` with the MAVLink message id 20. Since this is a manual MAVLink request command, it will only appear in the datastream when and where it is included in the MAVSDK mission code. The request is sent right after a connection with the drone is made, and before the mission is uploaded. When the request is sent, a message is sent to the overhead program through the terminal output which then is used to create a unified base-time t_0 . All timestamps are then set in reference to t_0 , as $t_0 + \Delta t$.

Once the packets were captured by sniffing the designated UDP port, they were saved to a PCAP file. Viewing the PCAP files in the terminal yielded a long list of data packets, each annotated with timestamps at microsecond precision. Figure 4 shows a single data packet where the top line includes the source and destination addresses, transport protocol (UDP), and payload length. Further beneath this summary the packet is displayed as a hexadecimal dump, where each row represents the raw byte values at specific offsets within the packet. This dump contains the complete packet structure, including the IP header, UDP header, and the application-layer payload.

As our adversary model assumes the adversary can only know the payload length and timestamp, the hexadecimal dump is out of the scope.

The PCAP files were processed based on the timestamps t_{event} for the specific mission, where all messages sent within $t_{event} \pm 1$ second of a timestamp were labeled with the timestamp event name. All messages sent from $> t_{event} \pm 2$ seconds from any timestamp were labeled as an "other" event. All messages that does not fit under these constraints are deleted. After this, the PCAP files were parsed into CSV files to easily debug with the first column being the timestamp and the second the payload size as each row is a packet. Every event is parsed as a group sequentially to retain its features in the time domain. A time series scatter plot was generated with all payload sizes at their arrival times, as indicated in Fig. 5.

E. Analysis

Through inspection of the packets and their timestamps over the time series, done at a smaller time interval and a higher precision of payload clusters (like in Fig. 6), two types of categories can be identified. Some messages are emitted constantly and with high frequency, primarily containing data such as odometry, heartbeat signals, or other semantics that a ground control station constantly needs. Meanwhile, some messages appear in bursts at specific timestamps which indicates that the messages are event-driven.

The visualized traffic event bursts appear as short intervals of increased packet density, often accompanied by the temporary appearance of additional packet sizes or changes in the relative frequency of existing ones. These bursts are likely associated with discrete behavioral transitions such as arming, takeoff preparation, landing, or mode changes. To understand when bursts likely occurred, the average inter-arrival time of each payload size and the standard deviation in a one second window was studied, as shown in Figure 7. This showed that at certain time windows there were bursts of messages.

The amount of unique payload sizes that were active were also studied which showed that in general, roughly the same amount of messages were active.

The lack of a consistent mapping between payload size and inter-arrival time demonstrates that communication patterns are not governed by simple deterministic relationships. Instead, behavior is encoded in subtle statistical variations, motivating the use of learning based approaches.

F. Feature Engineering

Sliding windows were used to segment the continuous packet stream into fixed-duration samples for supervised learning. Individual packets do not provide enough context to distinguish drone behavior; instead, each sample is represented by short-term timing and payload-size patterns within a fixed time interval.

1) *Sliding Window*: For each window, packets are sorted by timestamp. Let the timestamps be $\{t_1, t_2, \dots, t_n\}$ and the corresponding payload sizes be $\{s_1, s_2, \dots, s_n\}$. The inter-arrival time between consecutive packets is defined as

$$\Delta t_i = t_i - t_{i-1}, \quad i = 2, \dots, n. \quad (7)$$

TABLE I
SLIDING WINDOW CONFIGURATIONS EVALUATED FOR FEATURE EXTRACTION

Window Configuration	Window Size (s)	Overlap Size (s)
Short	0.5	0.40
Mid	1.0	0.80
Long	1.5	1.20

Different sliding window configurations were evaluated to balance temporal resolution and sample independence. The window size determines the temporal context of each sample, while the overlap size describes how much two consecutive windows overlap. The configurations are summarized in Table I.

2) Window-Level Features:

3) *Window-Level Features*: The first group of features describes the total traffic volume within a window. The packet count (`packet_count`) is the number of packets observed in the window, while the packet rate is defined as n/T and corresponds to `packet_rate`, where n is the number of packets and T is the window size. The observed window duration is computed as $t_n - t_1$ and corresponds to `window_duration`.

The payload-size distribution features are computed from the exact packet sizes observed within each window. Let c_k denote the number of packets with payload size k , and define $p_k = c_k/n$. The payload-size entropy is given by

$$H_s = - \sum_k p_k \log_2 p_k. \quad (8)$$

This corresponds to the feature `size_entropy`. Additional payload-size distribution features include the number of unique payload sizes (`unique_size_count`), the unique-size ratio (`unique_size_ratio`), and the dominant-size fraction (`dominant_size_fraction`). The dominant-size fraction measures the proportion of packets belonging to the most frequent payload size within the window.

Exact payload-size information is represented only through binary presence features, described below, rather than through separate normalized frequency features. This avoids redundant features such as both the frequency and presence of the same payload size.

4) *Timing Features*: Inter-arrival time statistics are computed from Δt_i . These include the mean (`mean_interarrival`), standard deviation (`std_interarrival`), variance (`var_interarrival`), minimum (`min_interarrival`), maximum (`max_interarrival`), and range (`interarrival_range`). Percentiles are represented as `p10_interarrival`, `p25_interarrival`, `p50_interarrival`, `p75_interarrival`, and `p90_interarrival`.

The coefficient of variation is defined as

$$CV_{\Delta t} = \frac{\sigma_{\Delta t}}{\mu_{\Delta t}}, \quad (9)$$

where Δt_i denotes inter-arrival times, and $\mu_{\Delta t}$ and $\sigma_{\Delta t}$ are their mean and standard deviation. This corresponds to the feature `interarrival_cv`.

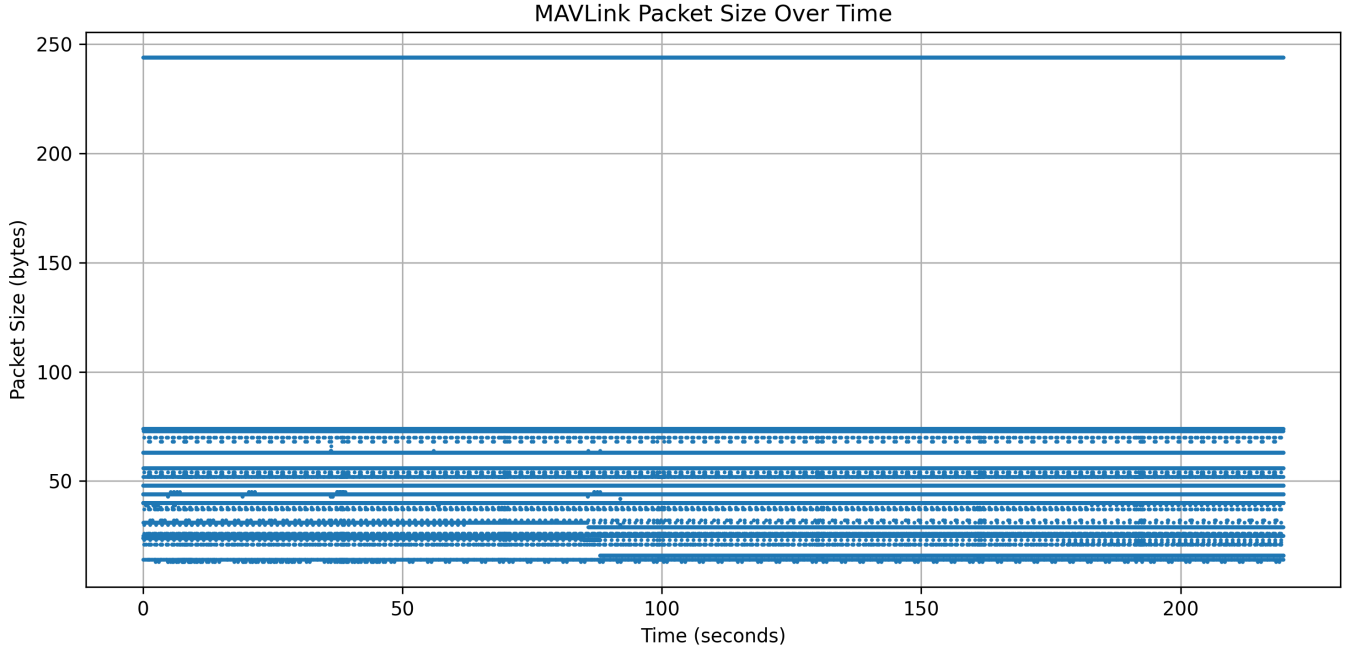


Fig. 5. Time series scatter plot of captured packet payload sizes over time.

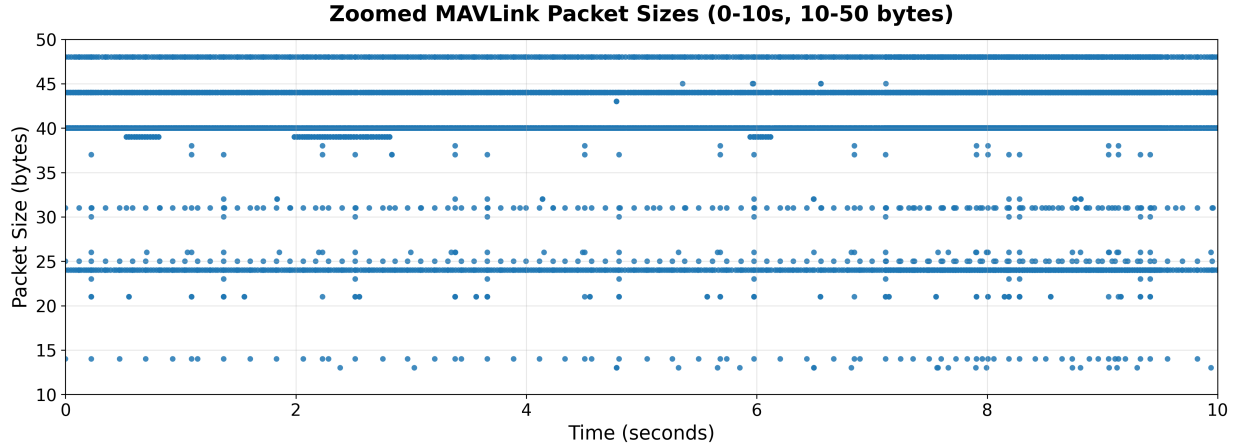


Fig. 6. Zoomed view of MAVLink packet sizes over a short time interval, highlighting clusters of frequently occurring packet sizes and sparse event-driven transmissions.

To capture timing irregularity, inter-arrival entropy (*interarrival_entropy*) is computed by binning the inter-arrival times and applying the entropy formula to the resulting distribution. Burst-related features describe uneven packet timing within the window. Let $\mu_{\Delta t}$ and $\sigma_{\Delta t}$ denote the mean and standard deviation of the inter-arrival times within a window. The burstiness index is defined as

$$B = \frac{\sigma_{\Delta t}}{\mu_{\Delta t} + \varepsilon}, \quad (10)$$

where ε is a small constant used to avoid division by zero. This feature is represented as *burstiness_index*. The silence fraction (*silence_fraction*) is the fraction of inter-arrival times above a silence threshold, while the rapid-gap fraction (*rapid_gap_fraction*) is the fraction below a rapid-gap threshold.

Temporal segmentation features are obtained by splitting each window into halves and thirds. For each segment, the fraction of packets occurring inside that segment is computed. These features include *first_half_packet_fraction*, *second_half_packet_fraction*, *first_third_packet_fraction*, *middle_third_packet_fraction*, and *last_third_packet_fraction*. These features capture how traffic is distributed over time inside the sliding window. Additional half-window features include *first_half_dominant_size_fraction*, *second_half_dominant_size_fraction*, *half_packet_count_diff*, and *half_dominant_size_diff*.

Sequential change features describe packet-

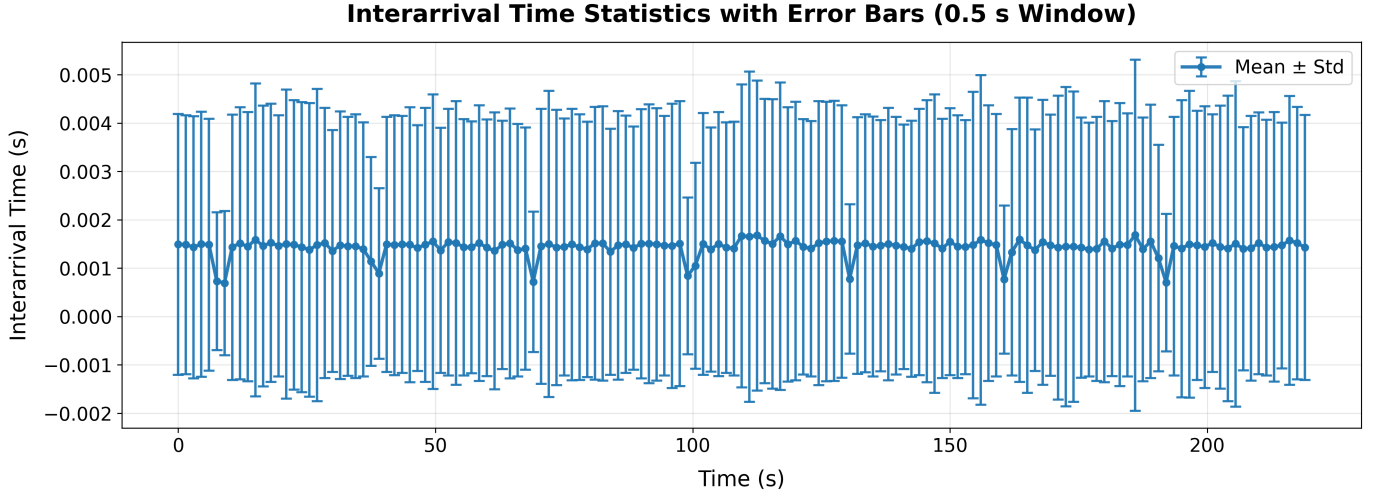


Fig. 7. Interarrival time statistics computed over sliding windows.

to-packet variations. These include the fraction of consecutive packets with identical sizes (`consecutive_same_size_fraction`), the mean and standard deviation of absolute payload-size changes (`mean_abs_size_change`, `std_abs_size_change`), and statistics of changes between consecutive inter-arrival times (`gap_change_mean_abs`, `gap_change_std`).

5) *Exact Packet Size Presence Features*: For each payload size k , a binary presence feature is defined as

$$\text{Presence}_k = \begin{cases} 1, & \text{if size } k \text{ appears in the window,} \\ 0, & \text{otherwise.} \end{cases} \quad (11)$$

These features capture rare but potentially informative packet sizes that may occur only once during a specific behavior. In the implementation, these features are named `size_present_X`, where X is the payload size in bytes. For example, `size_present_33` is equal to 1 if at least one 33-byte payload appears in the window, and 0 otherwise.

To make the feature-importance plots easier to interpret, the generated figures use descriptive labels rather than only internal implementation names. For example, `Payload-size presence: 33 bytes` corresponds to the feature `size_present_33`. Labels beginning with `Packet timing` describe statistics computed from inter-arrival times, labels beginning with `Payload-size distribution` describe aggregate properties of the payload-size distribution, labels beginning with `Window timing` describe where packets occur within the sliding window, and labels beginning with `Packet sequence` describe changes between consecutive packets. These labels do not introduce additional features; they are only clearer descriptions of the engineered features used by the model.

G. Model Selection and Training

There are different algorithms of supervised learning that can be applied to use the labeled data. The purpose of using different algorithms is to investigate whether sequential

TABLE II
GRU ARCHITECTURE

Layer	Type	Main setting
1	Input	(128, 3) sequence
2	Masking	pad value = -999
3	GRU	64 units
4	Dense	64 units, ReLU
5	Dropout	rate = 0.40
6	Output	Softmax

learning or learning from instance specific features is more important for inferring different drone operations.

Random Forest was used as a baseline model to see if the unsequenced windows, independent of each other, carry information. It is well-suited for this task because it can model nonlinear relationships between features and is robust to noise and irrelevant variables. The model was configured with 300 decision trees.

XGBoost was included as a more advanced tree-based method. Unlike Random Forest, which builds trees independently, XGBoost constructs trees sequentially where each tree focuses on correcting the errors of the previous ones. This makes it particularly effective for capturing subtle differences in traffic behavior, such as variations in packet timing and packet-size distributions. In this work, the model used 400 trees, a maximum depth of 5, and a learning rate of 0.03.

The GRU model was used to capture temporal dependencies in packet sequences. Instead of relying only on tabular features, this model processes the ordered sequence of packets within each window. The sequence was represented using packet timing and packet size, and padded to a fixed length. The architecture used is shown in Table II.

The CNN model was applied to the tabular feature vectors. The feature vector was reshaped to allow convolutional layers to operate across the feature dimension. The CNN architecture was designed to capture local interactions between features while keeping the model relatively simple. The architecture used is shown in Table III.

The neural network models were trained using the Adam

TABLE III
CNN ARCHITECTURE

Layer	Type	Main setting
1	Input	$(F, 1)$ feature vector
2	Conv1D	32 filters, kernel 3
3	Conv1D	32 filters, kernel 3
4	Global Max Pooling	feature reduction
5	Dense	64 units, ReLU
6	Dropout	rate = 0.30
7	Output	Softmax

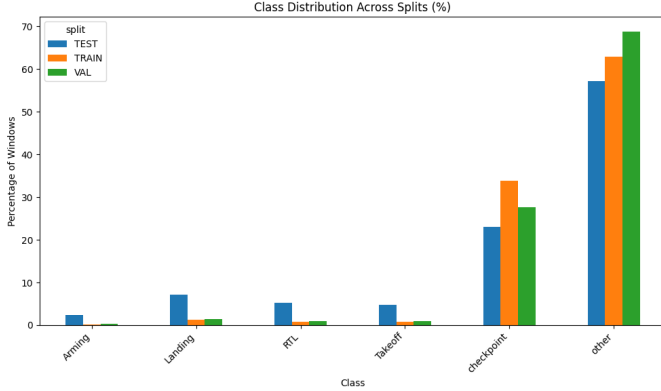


Fig. 8. Class distribution across the dataset. The “other” class dominates with approximately 63% of the samples.

optimizer and categorical cross-entropy loss. Early stopping was applied based on validation loss to prevent overfitting. Since the different drone event classes are imbalanced, where the “other” class is dominant as shown in Fig. 8, weights were used. The weights were assigned to each class inversely proportional to its relative frequency. For example, The “other” class has approximately 63% of the training data, resulting in a weight of about 0.26.

The dataset was split into training, validation, and test sets to evaluate model performance. The data was randomly fragmented at the mission file level to avoid overlap leakage between training and testing. The training set contained 9 mission files, the validation set contained 2 mission files, and the final test set contained 1 separate file.

To ensure consistency across experiments, all models were trained and evaluated on the same data splits. Performance was measured using standard classification metrics, including accuracy, precision, recall, and F1-score.

A purity filter was applied during window labeling. Each window was assigned the dominant label. However, it was only retained if at least 90% of the packets in the window belonged to that label; otherwise, the window was discarded to reduce label ambiguity.

V. RESULTS

Model performance across the three window configurations is summarized in Tables IV–VII. The best overall result was achieved by XGBoost using the long window configuration. The best result for each algorithm is highlighted in bold. In addition to the numerical model results, Fig. 10 and Fig. 11 provide further insight into how the models learned through

TABLE IV
CNN PERFORMANCE ACROSS WINDOW SETTINGS

Window	Acc.	F1
Short	0.6685	0.6443
Mid	0.6587	0.4886
Long	0.6323	0.5545

TABLE V
GRU PERFORMANCE ACROSS WINDOW SETTINGS

Window	Acc.	F1
Short	0.1326	0.1071
Mid	0.2452	0.1464
Long	0.4839	0.1515

TABLE VI
RANDOM FOREST PERFORMANCE ACROSS WINDOW SETTINGS

Window	Acc.	F1
Short	0.6768	0.5290
Mid	0.6635	0.5184
Long	0.6387	0.4389

TABLE VII
XGBOOST PERFORMANCE ACROSS WINDOW SETTINGS

Window	Acc.	F1
Short	0.6464	0.5762
Baseline	0.6875	0.6585
Long	0.7226	0.6598

epochs. Fig. 10 shows the most influential features for the best-performing models. Fig. 11 shows the training and validation behavior of XGBoost, CNN, and GRU, making it possible to compare how stable the learning process was across the different model types.

The normalized confusion matrices show in each row how each label is classified, dependent on the models.

VI. DISCUSSION

A. Interpretation of Results

XGBoost achieved the best performance, reaching an accuracy of 0.7226 and an F1-score of 0.6598 with the long window configuration, as shown in Table VII. This was higher than Random Forest, CNN, and GRU, indicating that boosted decision trees were most effective for capturing nonlinear relationships in the engineered traffic features.

The CNN achieved moderate performance, while the GRU performed worst across the tested configurations. This suggests that aggregated window-level statistics were more informative than the raw packet sequence representation used by the GRU.

With five classes, random guessing would give an expected accuracy of approximately 16.77%. Another baseline would be to only predict “Other” which would yield a accuracy of 55%. All models exceeded these baselines, but the confusion matrices in Fig. 9 show that several errors are related to confusion with the dominant “Other” class.

Window size also affected performance. XGBoost improved from 0.6464 accuracy with the short window to 0.7226 with the long window. This suggests that longer windows provide more stable traffic statistics, although they reduce temporal precision. The feature-importance plots in Fig. 10 show that

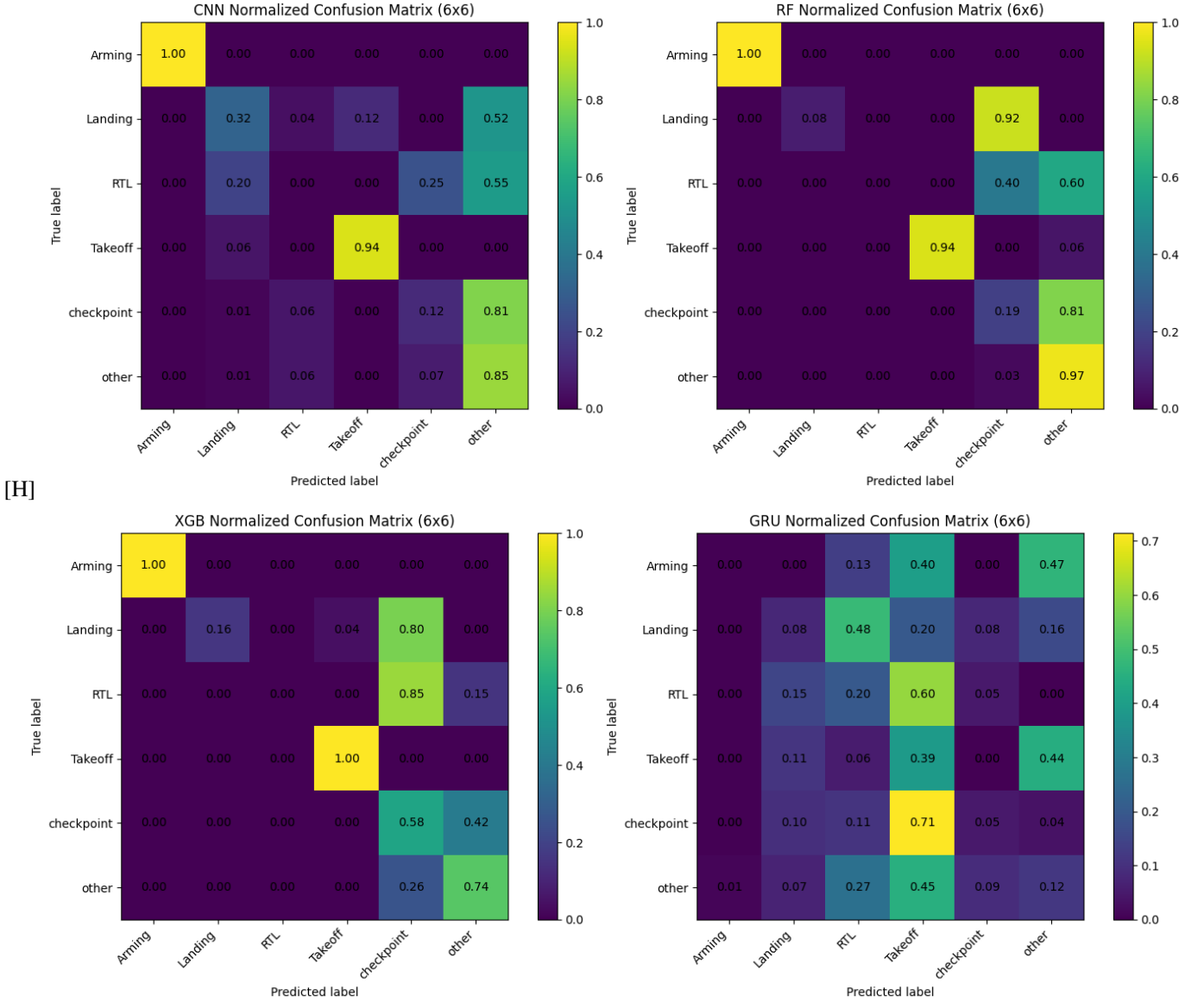


Fig. 9. Normalized confusion matrices for CNN, Random Forest, XGBoost, and GRU.

packet-size features dominate the rankings. This suggests that distinct MAVLink message sizes provide useful behavioral information, even when payload content is not inspected.

As mentioned previously, the window configuration changes the accuracy. As shown in Tables IV–VII, XGBoost improved from 0.6464 (short window) to 0.7226 (long window), indicating that larger windows provide more stable statistical patterns. Short windows introduce variability and noise, reducing performance, while longer windows improve reliability at the cost of temporal resolution.

The feature importance results in Fig. 10 show that payload size features dominate both native and permutation importance rankings. This suggests that specific packet sizes strongly encode behavior, likely due to MAVLink message structures where distinct messages have characteristic sizes. Timing-based features remain useful but are less influential.

As shown in Fig. 11, the GRU model displays unstable

accuracy across epochs while the loss remains high, indicating that the sequence-based model is underfitting or learning very little from the raw packet sequence representation. This suggests that the current GRU input representation does not capture the relevant traffic patterns as effectively as the engineered window-level features used by XGBoost, Random Forest, and CNN. To improve performance, more training data could be generated, and the sequence representation could be redesigned to better capture packet timing and payload-size transitions. Unsupervised learning could also assist in creating or refining labels by identifying whether naturally occurring traffic clusters correspond to drone behaviors, or whether confused classes are statistically similar.

B. Defense Strategies and Mitigations

Many mitigations can be implemented on protocol level, that generally defend against traffic analyses by minimizing

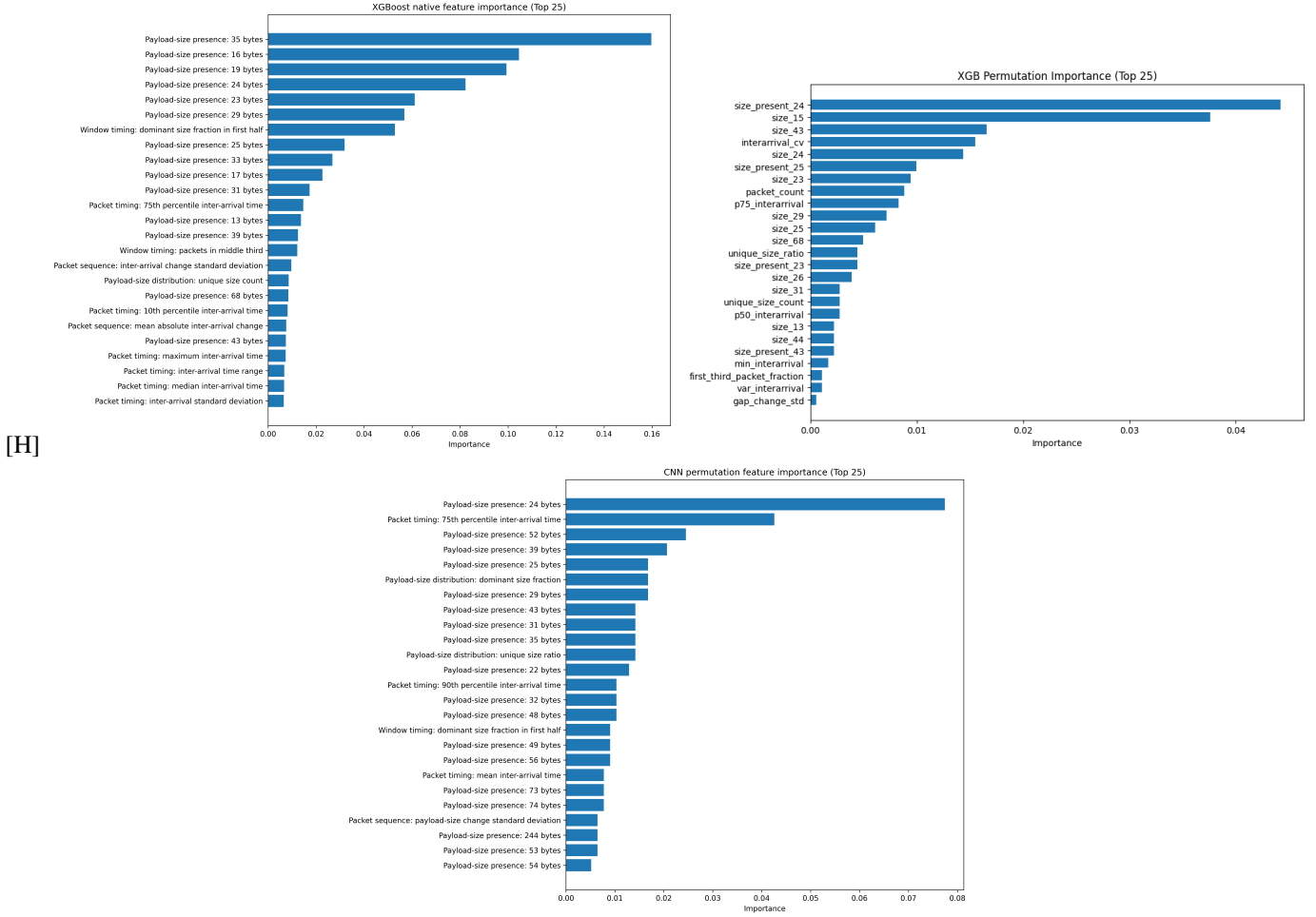


Fig. 10. Feature importance results for XGBoost and CNN.

the amount of discriminant behavior [13].

These clear discriminant signals could be hidden through a number of ways, such as noise injection in the serialized data or frequency jumping of the transmission signal - this would, however, be computationally heavy which would defeat the purpose of a lightweight telemetry protocol. Simple mitigation strategies specific to our contribution is combining certain payload sized messages into common clusters, as the size related feature were the most useful for classifying. Another way to confuse the model could be to randomize certain message sizes and inform the ground control station what sizes messages will be, through a handshake. When an adversary cannot infer the drone state based on payload sizes, inferences would be based on time-based features. However, as mentioned above, the real-time constraints needed for UAVs in a lightweight link must be considered. Overall the result conclude that there is viable information that can be gathered by an adversary purely by statistical analysis of communication. This information can play a vital part in a larger setting - by knowing the current and preceding events of an AV, deductions can be made about its purpose, state and mission. If, for example, there is a drone that at certain regular intervals crosses checkpoints the deduction can be made that there is a drone searching or mapping out a grid-pattern. A sudden switch to an offboard mode would

then indicate that a point of interest has been found, and in opposite a Return to Launch-event would indicate that nothing of interest has been found. This clearly demonstrates that the information gathered through an eavesdropping attack can have greater ramifications than obvious at first glance. What these effects are depends on context, whether it is civilian or military, but in both cases the need for further development of native end-to-end encryption is clear.

CONCLUSION

We have investigated the feasibility of side channel attacks on drones using the MAVLink telemetry protocol. By training a supervised model on data from simulated drone flights, we have proved the feasibility of our concept with an event classification accuracy of 72%. This implies that all telemetry protocols with a distinct traffic behaviour is vulnerable, even with added encryption. This is further reinforced by the finding that size related features carries the greatest significance in classifying events, making predefined payload sizes a security issue that need to be addressed. Future work that can be derived from these results could be in further developing our supervised model to work in real-time on real-world captured drone data, further proving the concept.

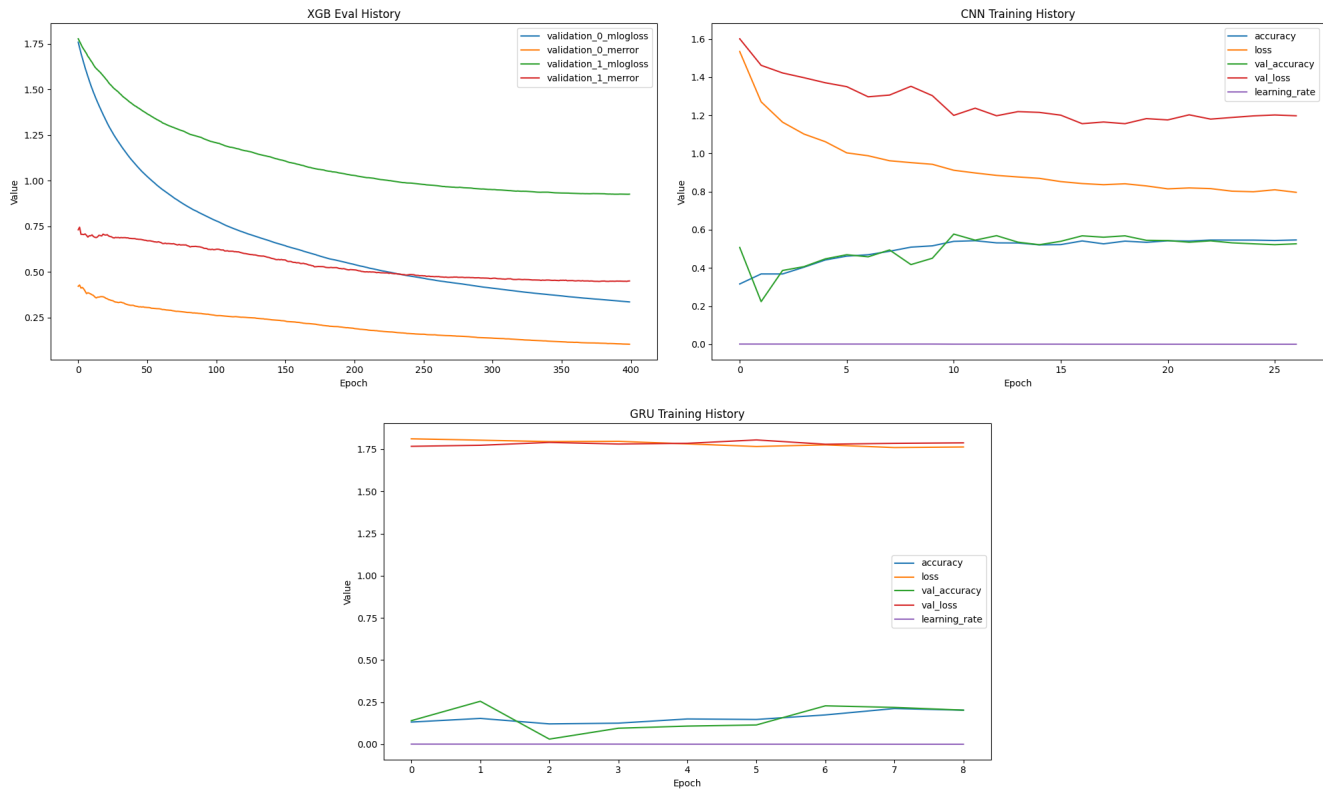


Fig. 11. Training and validation performance for XGBoost, CNN, and GRU.

ACKNOWLEDGMENT

The authors would like to thank our supervisor Axel Andersson for his guidance and feedback throughout the completion of this thesis.

REFERENCES

- [1] Dronecode. (2026, Apr.) Open source autopilot for drone developers. [Online]. Available: <https://px4.io/>
- [2] —. (2026, Apr.) Qgroundcontrol. [Online]. Available: <https://qgroundcontrol.com/>
- [3] M. D. Team. (2026, Feb.) Mavlinkv2. [Online]. Available: https://mavlink.io/en/guide/mavlink_2.html
- [4] A. Erba, J. H. Castellanos, S. Sihag, S. Zonouz, and N. O. Tippenhauer, "Sensor deprivation attacks for stealthy uav manipulation," 2024. [Online]. Available: <https://arxiv.org/abs/2410.11131>
- [5] A. Koubâa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith, and M. Khalgui, "Micro air vehicle link (mavlink) in a nutshell: A survey," *IEEE Access*, vol. 7, pp. 87 658–87 680, 2019.
- [6] Dronecode. (2026, Apr.) Packet serialization. [Online]. Available: <https://mavlink.io/en/guide/serialization.html>
- [7] M. Veksler, K. Akkaya, and S. Uluagac, "Catch me if you can: Covert information leakage from drones using mavlink protocol," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, ser. ASIA CCS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 902–914. [Online]. Available: <https://doi.org/10.1145/3634737.3637672>
- [8] M. Lotfollahi, R. Shirali Hossein Zade, M. J. Siavoshani, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *arXiv preprint arXiv:1709.02656*, 2017.
- [9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [10] O. Robotics. (2026, Apr.) About gazebo. [Online]. Available: <https://gazebo.org/about>
- [11] Dronecode. (2026, Apr.) Mavsdk. [Online]. Available: <https://docs.px4.io/main/en/robotics/mavsdk>
- [12] J. Postel, "User datagram protocol," Internet Engineering Task Force, Tech. Rep. RFC 768, Aug. 1980. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc768>
- [13] R. Anderson, *Security Engineering: A Guide to Building Dependable Distributed Systems*. Wiley, 2008.